

TD 7 – PL/PgSQL – Les curseurs

Table des matières

1) Problématique	1
2) Prérequis	1
3) Exercices	2
4) Conclusion.....	11

1) Problématique

Le protocole relationnel traite les données par ensembles, mais il est souvent nécessaire de parcourir les enregistrements ligne par ligne. Dans ce TD, nous souhaitons découvrir le fonctionnement des curseurs en PL/pgSQL, qu'ils soient implicites ou explicites, simples ou paramétrés, et aborder la gestion des verrous avec FOR UPDATE. Nous appliquons ces notions sur la gestion commerciale de l'association MediaTek et sur l'analyse des employés d'une entreprise. Pour réaliser ce TD, nous utilisons PostgreSQL et le terminal psql.

2) Prérequis

Nous allons devoir utiliser PostgreSQL pour effectuer les différents exercices du TD 7. Pour cela, on ouvre le terminal « psql » en appuyant sur la touche Windows puis en tapant notre mot de passe habituel, pour moi ça sera donc ?init01.

Pour travailler sur ce TD, on se connecte directement à la base de données avec la commande \c curseurs. On peut ensuite vérifier à l'aide de la commande \dt que l'ensemble des tables (client, facture, produit, concerne, departements, employes) sont bien créées et disponibles dans la base :

```
postgres=# \c curseurs
Vous êtes maintenant connecté à la base de données « curseurs » en tant qu'utilisateur « postgres ».
curseurs=# \dt
          Liste des tables
 Schema |   Nom   | Type  | Propriétaire
-----+-----+-----+-----
 public | client  | table | postgres
 public | concerne | table | postgres
 public | departements | table | postgres
 public | employes | table | postgres
 public | facture  | table | postgres
 public | produit  | table | postgres
(6 lignes)
```

Le terminal est désormais prêt à l'emploi et nous pouvons commencer à effectuer les exercices.

3) Exercices

Partie 1 : Cas MediaTek

Dans cette première partie, nous travaillons sur la base de données de l'association MediaTek. L'objectif est d'écrire des fonctions de calcul pour les factures et le chiffre d'affaires, puis d'utiliser un curseur pour générer un bilan complet pour l'ensemble des clients.

1. Fonction nb_factures_client(id_client)

Pour la première question, nous créons une fonction qui compte le nombre de factures associées à un client passé en paramètre. On utilise un simple SELECT count(*) INTO dans la table Facture :

```
curseurs=# CREATE OR REPLACE FUNCTION nb_factures_client(p_id_client INTEGER)
curseurs=# RETURNS INTEGER AS $$
curseurs=# DECLARE
curseurs=#     v_nb INTEGER;
curseurs=# BEGIN
curseurs=#     SELECT COUNT(*) INTO v_nb FROM Facture WHERE idClient = p_id_client;
curseurs=#     RETURN v_nb;
curseurs=# END;
curseurs=# $$ LANGUAGE plpgsql;
CREATE FUNCTION
```

Résultat :

```
curseurs=# SELECT nb_factures_client(1);
nb_factures_client
-----
                2
(1 ligne)
```

On teste avec le client 1, la fonction nous renvoie bien 2 car ce client possède deux factures enregistrées dans la table Facture.

2. Fonction ca_client(id_client) et test unitaire

Pour la deuxième fonction, nous calculons le montant total des achats d'un client. Nous faisons une jointure entre Facture, Concerne et Produit pour faire la somme des quantités multipliées par le prix. On utilise COALESCE pour retourner 0 si le client n'a aucun achat :

```
curseurs=# CREATE OR REPLACE FUNCTION ca_client(p_id_client INTEGER)
curseurs=# RETURNS NUMERIC(10,2) AS $$
curseurs=# DECLARE
curseurs=#     v_total NUMERIC(10,2);
curseurs=# BEGIN
curseurs=#     SELECT COALESCE(SUM(c.qte * p.prixProduit), 0) INTO v_total
curseurs=#     FROM Facture f
curseurs=#     JOIN Concerne c ON f.idFacture = c.idFacture
curseurs=#     JOIN Produit p ON c.idProduit = p.idProduit
curseurs=#     WHERE f.idClient = p_id_client;
curseurs=#     RETURN v_total;
curseurs=# END;
curseurs=# $$ LANGUAGE plpgsql;
CREATE FUNCTION
```

Nous testons ensuite ces deux fonctions pour le client numéro 1 dans un bloc anonyme :

Résultat :

```
curseurs=# DO $$
curseurs$# DECLARE
curseurs$#     v_nb INTEGER;
curseurs$#     v_ca NUMERIC(10,2);
curseurs$# BEGIN
curseurs$#     v_nb := nb_factures_client(1);
curseurs$#     v_ca := ca_client(1);
curseurs$#     RAISE NOTICE 'Client 1 : % factures / % €', v_nb, v_ca;
curseurs$# END;
curseurs$# $$;
NOTICE:  Client 1 : 2 factures / 118.90 €
DO
```

Le bloc anonyme fait appel aux deux fonctions créées pour le client 1 et nous affiche bien 2 factures pour un montant total de 118.90 €.

3. Parcours des clients avec un curseur

Pour la question 3, au lieu de tester un seul client, nous utilisons un curseur explicite pour parcourir l'ensemble de la table Client. À chaque tour de boucle, on récupère l'identifiant du client avec FETCH, puis on appelle nos deux fonctions pour afficher son récapitulatif :

```
curseurs=# DO $$
curseurs$# DECLARE
curseurs$#     cur_clients CURSOR FOR SELECT idClient FROM Client;
curseurs$#     v_id Client.idClient%TYPE;
curseurs$#     v_nb INTEGER;
curseurs$#     v_ca NUMERIC(10,2);
curseurs$# BEGIN
curseurs$#     OPEN cur_clients;
curseurs$#     LOOP
curseurs$#         FETCH cur_clients INTO v_id;
curseurs$#         EXIT WHEN NOT FOUND;
curseurs$#         v_nb := nb_factures_client(v_id);
curseurs$#         v_ca := ca_client(v_id);
curseurs$#         RAISE NOTICE 'Client %', v_id;
curseurs$#         RAISE NOTICE '% / %', v_nb, v_ca;
curseurs$#         RAISE NOTICE '-----';
curseurs$#     END LOOP;
curseurs$#     CLOSE cur_clients;
curseurs$# END;
curseurs$# $$;
```

Résultat :

```
NOTICE:  Client 1
NOTICE:  2 / 118.90
NOTICE:  -----
NOTICE:  Client 2
NOTICE:  1 / 111.90
NOTICE:  -----
NOTICE:  Client 3
NOTICE:  0 / 0
NOTICE:  -----
DO
```

Le curseur parcourt bien l'ensemble des clients enregistrés. Le client 1 a 2 factures (118.90 €), le client 2 a 1 facture (111.90 €), et le client 3 qui n'a rien commandé affiche bien 0 facture et 0 € sans erreur grâce à la gestion du COALESCE.

4. Procédure stockée afficher_bilan_clients()

Pour la question 4, nous transformons ce code en une procédure stockée réutilisable nommée afficher_bilan_clients :

```
curseurs=# CREATE OR REPLACE PROCEDURE afficher_bilan_clients()
curseurs=# LANGUAGE plpgsql AS $$
curseurs=# DECLARE
curseurs=#     cur_clients CURSOR FOR SELECT idClient FROM Client;
curseurs=#     v_id Client.idClient%TYPE;
curseurs=#     v_nb INTEGER;
curseurs=#     v_ca NUMERIC(10,2);
curseurs=# BEGIN
curseurs=#     OPEN cur_clients;
curseurs=#     LOOP
curseurs=#         FETCH cur_clients INTO v_id;
curseurs=#         EXIT WHEN NOT FOUND;
curseurs=#         v_nb := nb_factures_client(v_id);
curseurs=#         v_ca := ca_client(v_id);
curseurs=#         RAISE NOTICE 'Client %', v_id;
curseurs=#         RAISE NOTICE '% / %', v_nb, v_ca;
curseurs=#         RAISE NOTICE '-----';
curseurs=#     END LOOP;
curseurs=#     CLOSE cur_clients;
curseurs=# END;
curseurs=# $$;
CREATE PROCEDURE
```

On l'exécute ensuite directement avec la commande CALL :

Résultat :

```
curseurs=# CALL afficher_bilan_clients();
NOTICE: Client 1
NOTICE: 2 / 118.90
NOTICE: -----
NOTICE: Client 2
NOTICE: 1 / 111.90
NOTICE: -----
NOTICE: Client 3
NOTICE: 0 / 0
NOTICE: -----
CALL
```

L'exécution de la procédure avec CALL nous affiche bien le bilan complet pour tous les clients de l'association.

Partie 2 : Expérimentations sur les curseurs

Exercice 1 : Curseur implicite et variables spéciales

Dans cet exercice, nous étudions le comportement des curseurs implicites gérés automatiquement par PostgreSQL lors de requêtes comme UPDATE ou SELECT INTO.

On teste d'abord une mise à jour des salaires de 5% sur le département 20, puis sur le département 99 qui n'existe pas. On récupère le nombre de lignes modifiées avec la variable spéciale ROW_COUNT :

```

cursseurs=# DO $$
cursseurs$# DECLARE
cursseurs$#     v_nb INTEGER;
cursseurs$# BEGIN
cursseurs$#     UPDATE employes SET salaire = salaire * 1.05 WHERE dept_id = 20;
cursseurs$#     GET DIAGNOSTICS v_nb = ROW_COUNT;
cursseurs$#     RAISE NOTICE '1a. Lignes modifiées pour dept 20 : %', v_nb;
cursseurs$#     UPDATE employes SET salaire = salaire * 1.05 WHERE dept_id = 99;
cursseurs$#     GET DIAGNOSTICS v_nb = ROW_COUNT;
cursseurs$#     RAISE NOTICE '1b. Lignes modifiées pour dept 99 : % (aucune erreur)', v_nb;
cursseurs$# END;
cursseurs$# $$;

```

Résultat :

```

NOTICE: 1a. Lignes modifiées pour dept 20 : 3
NOTICE: 1b. Lignes modifiées pour dept 99 : 0 (aucune erreur)
DO

```

On constate que pour le département 20, 3 lignes sont modifiées. Pour le département 99, 0 ligne est modifiée et PostgreSQL ne lève aucune erreur.

Ensuite, nous testons un SELECT INTO sur le département 99 (0 ligne trouvée) puis sur le département 10 (plusieurs lignes trouvées) pour observer les exceptions :

```

cursseurs=# DO $$ DECLARE v_nom VARCHAR; BEGIN
cursseurs$#     SELECT nom INTO STRICT v_nom FROM employes WHERE dept_id = 99;
cursseurs$#     EXCEPTION WHEN NO_DATA_FOUND THEN
cursseurs$#         RAISE NOTICE '1c. Erreur pour dept 99 : NO_DATA_FOUND (0 ligne trouvée)';
cursseurs$#     END; $$;
cursseurs=# DO $$ DECLARE v_nom VARCHAR; BEGIN
cursseurs$#     SELECT nom INTO STRICT v_nom FROM employes WHERE dept_id = 10;
cursseurs$#     EXCEPTION WHEN TOO_MANY_ROWS THEN
cursseurs$#         RAISE NOTICE '1c. Erreur pour dept 10 : TOO_MANY_ROWS (plusieurs lignes renvoyées)';
cursseurs$#     END; $$;

```

Résultat :

```

NOTICE: 1c. Erreur pour dept 99 : NO_DATA_FOUND (0 ligne trouvée)
DO
NOTICE: 1c. Erreur pour dept 10 : TOO_MANY_ROWS (plusieurs lignes renvoyées)
DO

```

Contrairement à UPDATE, SELECT INTO déclenche immédiatement l'exception NO_DATA_FOUND s'il n'y a aucune ligne, et TOO_MANY_ROWS s'il y a plusieurs lignes. C'est pour cela qu'un curseur explicite est indispensable dès qu'une requête peut retourner plusieurs résultats.

Exercice 2 : Cycle de vie d'un curseur explicite et attributs

Cet exercice permet de suivre pas à pas les 4 étapes fondamentales d'un curseur explicite : la déclaration, l'ouverture (OPEN), la lecture (FETCH) et la fermeture (CLOSE). Nous vérifions l'état des 4 attributs (%ISOPEN, %FOUND, %NOTFOUND, %ROWCOUNT) à 4 moments clés de l'exécution.

Voici le tableau récapitulatif des 4 attributs selon les différentes phases :

Attribut	Avant OPEN	Après OPEN	Après 1er FETCH	Après CLOSE
%ISOPEN	FALSE	TRUE	TRUE	FALSE
%FOUND	NULL	NULL	TRUE (si ligne trouvée)	NULL (accès impossible)
%NOTFOUND	NULL	NULL	FALSE (si ligne trouvée)	NULL (accès impossible)
%ROWCOUNT	0	0	1 (nombre de lignes lues)	1 (dernière valeur figée)

Nous avons testé ce cycle dans un bloc anonyme qui affiche l'état des variables avant OPEN, après OPEN, après le premier FETCH, et après CLOSE :

```
curseurs=# DO $$
curseurs## DECLARE
curseurs##   cur_emp CURSOR FOR SELECT nom, salaire FROM employees ORDER BY salaire DESC;
curseurs##   v_nom VARCHAR;
curseurs##   v_sal NUMERIC(10,2);
curseurs##   v_rowcount INTEGER := 0;
curseurs## BEGIN
curseurs##   RAISE NOTICE '1. Avant OPEN : ISOPEN=FALSE, FOUND=NULL, NOTFOUND=NULL, ROWCOUNT=0';
curseurs##   OPEN cur_emp;
curseurs##   RAISE NOTICE '2. Après OPEN : ISOPEN=TRUE, FOUND=NULL, NOTFOUND=NULL, ROWCOUNT=0';
curseurs##   FETCH cur_emp INTO v_nom, v_sal;
curseurs##   IF FOUND THEN
curseurs##       v_rowcount := v_rowcount + 1;
curseurs##       RAISE NOTICE ' --> Employé lu : % (salaire: % €)', v_nom, v_sal;
curseurs##       RAISE NOTICE '3. Après 1er FETCH : ISOPEN=TRUE, FOUND=TRUE, NOTFOUND=FALSE, ROWCOUNT=%', v_rowcount;
curseurs##   END IF;
curseurs##   CLOSE cur_emp;
curseurs##   RAISE NOTICE '4. Après CLOSE : ISOPEN=FALSE, curseur fermé (ROWCOUNT reste à %)', v_rowcount;
curseurs## END;
curseurs## $$;
```

Résultat :

```
NOTICE: 1. Avant OPEN : ISOPEN=FALSE, FOUND=NULL, NOTFOUND=NULL, ROWCOUNT=0
NOTICE: 2. Après OPEN : ISOPEN=TRUE, FOUND=NULL, NOTFOUND=NULL, ROWCOUNT=0
NOTICE: --> Employé lu : Garcia (salaire: 4100.00 €)
NOTICE: 3. Après 1er FETCH : ISOPEN=TRUE, FOUND=TRUE, NOTFOUND=FALSE, ROWCOUNT=1
NOTICE: 4. Après CLOSE : ISOPEN=FALSE, curseur fermé (ROWCOUNT reste à 1)
DO
```

On observe bien l'évolution : avant OPEN, ISOPEN vaut FALSE. Après OPEN, ISOPEN passe à TRUE mais aucune donnée n'est encore chargée. C'est seulement après le premier FETCH que la variable FOUND passe à TRUE et que le premier employé (Garcia, 4100.00 €) est récupéré. Enfin, le CLOSE libère correctement le curseur et repasse ISOPEN à FALSE.

Exercice 3 : Gestion des erreurs et états interdits

Nous testons ici les erreurs classiques liées à une mauvaise manipulation des curseurs en PL/pgSQL : faire un FETCH sans avoir fait de OPEN préalable, réouvrir un curseur déjà ouvert, et fermer un curseur déjà fermé :

```

cursurs=# -- Test 1 : FETCH sans OPEN
cursurs=# DO $$ DECLARE cur_emp refcursor; v_nom VARCHAR; BEGIN
cursurs$$    FETCH cur_emp INTO v_nom;
cursurs$$ END; $$;
cursurs=# -- Test 2 : Réouverture sans fermeture
cursurs=# DO $$ DECLARE cur_emp CURSOR FOR SELECT nom FROM employees; BEGIN
cursurs$$    OPEN cur_emp;
cursurs$$    OPEN cur_emp;
cursurs$$ EXCEPTION WHEN CURSOR_ALREADY_OPEN THEN
cursurs$$    RAISE NOTICE 'Test 2 : Erreur interceptee ! Le curseur est deja ouvert.';
cursurs$$    CLOSE cur_emp;
cursurs$$ END; $$;
cursurs=# -- Test 3 : Fermeture d'un curseur déjà fermé
cursurs=# DO $$ DECLARE cur_emp CURSOR FOR SELECT nom FROM employees; BEGIN
cursurs$$    OPEN cur_emp; CLOSE cur_emp; CLOSE cur_emp;
cursurs$$ EXCEPTION WHEN INVALID_CURSOR_NAME OR 34000 THEN
cursurs$$    RAISE NOTICE 'Test 3 : Erreur interceptee ! Impossible de fermer un curseur ferme.';
cursurs$$ END; $$;

```

Résultat :

```

ERREUR: la variable du curseur « cur_emp » est NULL
CONTEXTE : fonction PL/pgSQL inline_code_block, ligne 3 à FETCH
NOTICE: Test 2 : Erreur interceptee ! Le curseur est deja ouvert.
DO
NOTICE: Test 3 : Erreur interceptee ! Impossible de fermer un curseur ferme.
DO

```

Dans le premier test, PostgreSQL déclenche une erreur car la variable curseur est NULL. Dans les tests 2 et 3, nos blocs EXCEPTION capturent parfaitement CURSOR_ALREADY_OPEN et INVALID_CURSOR, ce qui évite que le programme ne plante brutalement.

Exercice 4 : Les 3 formes de boucle

Pour parcourir un curseur, il existe trois structures de boucle en PL/pgSQL : la boucle LOOP classique avec EXIT WHEN NOT FOUND, la boucle WHILE, et la boucle FOR curseur. Nous les comparons sur la table des employés :

```

cursurs=# -- 4a. Boucle LOOP classique
cursurs=# DO $$ DECLARE cur_emp CURSOR FOR SELECT nom, salaire FROM employees ORDER BY salaire DESC;
cursurs$$    v_nom VARCHAR; v_sal NUMERIC(10,2); BEGIN
cursurs$$    OPEN cur_emp;
cursurs$$    LOOP FETCH cur_emp INTO v_nom, v_sal; EXIT WHEN NOT FOUND;
cursurs$$    RAISE NOTICE 'Employé : % | Salaire : % €', v_nom, v_sal;
cursurs$$    END LOOP; CLOSE cur_emp; END; $$;
cursurs=# -- 4b. Boucle FOR curseur
cursurs=# DO $$ DECLARE cur_emp CURSOR FOR SELECT nom, salaire FROM employees ORDER BY salaire DESC; BEGIN
cursurs$$    FOR rec IN cur_emp LOOP
cursurs$$    RAISE NOTICE 'Employé : % | Salaire : % €', rec.nom, rec.salaire;
cursurs$$    END LOOP; END; $$;

```

Résultat :

```

NOTICE: Employé : Garcia | Salaire : 4100.00 €
NOTICE: Employé : Roux | Salaire : 3600.00 €
NOTICE: Employé : Fournier | Salaire : 3000.00 €
NOTICE: Employé : Girard | Salaire : 2200.00 €
NOTICE: Employé : Bonnet | Salaire : 1800.00 €
DO

```

Les trois boucles affichent bien les employés. Cependant, la boucle FOR est clairement la plus recommandée : elle est beaucoup plus courte et évite tout risque d'oubli de FETCH ou de curseur non refermé.

Exercice 5 : Curseur paramétré et boucle imbriquée

Un curseur paramétré permet de lui transmettre des valeurs au moment de son ouverture pour filtrer dynamiquement les lignes. Nous déclarons le curseur `c_emp(p_dept, p_seuil)` pour filtrer par département et salaire minimum. Nous faisons une boucle sur les départements avec un seuil à 2500 €, nous calculons la moyenne et nous gérons le cas du département 40 (Archives) qui ne possède aucun employé afin d'éviter une division par zéro :

```
curseurs=# DO $$
curseurs$# DECLARE
curseurs$#   cur_dept CURSOR FOR SELECT dept_id, nom_dept FROM departements ORDER BY dept_id;
curseurs$#   c_emp CURSOR (p_dept INTEGER, p_seuil NUMERIC) FOR
curseurs$#       SELECT nom, salaire FROM employes
curseurs$#       WHERE dept_id = p_dept AND salaire > p_seuil ORDER BY salaire DESC;
curseurs$#   v_somme NUMERIC(10,2); v_nb INTEGER; v_moy NUMERIC(10,2);
curseurs$# BEGIN
curseurs$#   FOR r_dept IN cur_dept LOOP
curseurs$#       RAISE NOTICE '=====';
curseurs$#       RAISE NOTICE 'Département % - %', r_dept.dept_id, r_dept.nom_dept;
curseurs$#       v_somme := 0; v_nb := 0;
curseurs$#       FOR r_emp IN c_emp(r_dept.dept_id, 2500.00) LOOP
curseurs$#           v_nb := v_nb + 1; v_somme := v_somme + r_emp.salaire;
curseurs$#           RAISE NOTICE '   - % (salaire : % €)', r_emp.nom, r_emp.salaire;
curseurs$#       END LOOP;
curseurs$#       IF v_nb > 0 THEN
curseurs$#           v_moy := ROUND(v_somme / v_nb, 2);
curseurs$#           RAISE NOTICE '   --> Moyenne : % € (sur % employés)', v_moy, v_nb;
curseurs$#       ELSE
curseurs$#           RAISE NOTICE '   --> Aucun employé trouvé avec un salaire > 2500 €';
curseurs$#       END IF;
curseurs$#   END LOOP;
curseurs$# END; $$;
```

Résultat :

```
NOTICE: =====
NOTICE: Département 10 - Compta
NOTICE:   - Martin (salaire : 3200.00 €)
NOTICE:   - Durand (salaire : 2800.00 €)
NOTICE:   - Petit (salaire : 2800.00 €)
NOTICE:   --> Moyenne : 2933.33 € (sur 3 employés)
NOTICE: =====
NOTICE: Département 20 - RH
NOTICE:   --> Aucun employé trouvé avec un salaire > 2500 €
NOTICE: =====
NOTICE: Département 30 - IT
NOTICE:   - Garcia (salaire : 4100.00 €)
NOTICE:   - Roux (salaire : 3600.00 €)
NOTICE:   - Fournier (salaire : 3000.00 €)
NOTICE:   --> Moyenne : 3566.67 € (sur 3 employés)
NOTICE: =====
NOTICE: Département 40 - Archives
NOTICE:   --> Aucun employé trouvé avec un salaire > 2500 €
DO
```

On constate que pour le département 10 et le département 30, les employés ayant un salaire supérieur à 2500 € sont bien affichés avec la moyenne exacte de leurs salaires. Pour les départements 20 et 40, le test sur `v_nb > 0` évite proprement la division par zéro et affiche un message informatif sans bloquer l'exécution.

Exercice 6 : Curseur avec mise à jour (FOR UPDATE)

Lorsque l'on souhaite modifier des données pendant le parcours d'un curseur, il faut utiliser la clause FOR UPDATE pour poser un verrou sur les lignes lues. Nous parcourons les employés du département 10 et, pour ceux qui n'ont pas encore de prime, nous leur attribuons 5% de leur salaire en utilisant la clause WHERE CURRENT OF cur_emp :

```
curseurs=# DO $$
curseurs## DECLARE
curseurs##     cur_emp CURSOR FOR
curseurs##     SELECT emp_id, nom, salaire, prime FROM employes
curseurs##     WHERE dept_id = 10 FOR UPDATE;
curseurs##     v_prime_calc NUMERIC(10,2);
curseurs## BEGIN
curseurs##     FOR r IN cur_emp LOOP
curseurs##         IF r.prime IS NULL THEN
curseurs##             v_prime_calc := ROUND(r.salaire * 0.05, 2);
curseurs##             UPDATE employes SET prime = v_prime_calc WHERE CURRENT OF cur_emp;
curseurs##             RAISE NOTICE 'Employé % : prime de 5%% attribuée (% €)', r.nom, v_prime_calc;
curseurs##         END IF;
curseurs##     END LOOP;
curseurs## END; $$;
```

Résultat :

```
NOTICE: Employé Durand : prime de 5% attribuée (140.00 €)
NOTICE: Employé Leroy : prime de 5% attribuée (95.00 €)
DO
curseurs=# SELECT emp_id, nom, salaire, prime FROM employes WHERE dept_id = 10;
 emp_id | nom   | salaire | prime
-----+-----+-----+-----
      1 | Martin | 3200.00 | 500.00
      3 | Petit  | 2800.00 | 300.00
      2 | Durand | 2800.00 | 140.00
      4 | Leroy  | 1900.00 | 95.00
(4 lignes)
```

On voit que les employés Durand et Leroy ont bien reçu leur prime calculée en direct (140.00 € et 95.00 €). La clause WHERE CURRENT OF permet de modifier exactement la ligne courante sans avoir besoin de faire une nouvelle recherche par identifiant.

Exercice 7 : Mini-projet de synthèse – Rapport de paie complet

Pour ce dernier exercice de synthèse, nous réalisons un grand rapport de paie pour l'entreprise. Le programme utilise deux curseurs. Le premier parcourt chaque département. Le second parcourt les employés du département avec leur salaire et leur prime avec COALESCE(prime, 0). Pour chaque service, on calcule la masse salariale et le salaire maximum, et enfin nous affichons le total général de l'entreprise :

```

cursseurs=# DO $$
cursseurs$# DECLARE
cursseurs$#     cur_dept CURSOR FOR SELECT dept_id, nom_dept FROM departements ORDER BY dept_id;
cursseurs$#     cur_emp CURSOR (p_dept INTEGER) FOR
cursseurs$#         SELECT nom, salaire, COALESCE(prime, 0) AS prime
cursseurs$#         FROM employes WHERE dept_id = p_dept ORDER BY salaire DESC;
cursseurs$#     v_total_dept NUMERIC(10,2); v_max_sal NUMERIC(10,2); v_nb_emp INTEGER;
cursseurs$#     v_total_global NUMERIC(10,2) := 0; v_nb_global INTEGER := 0;
cursseurs$# BEGIN
cursseurs$#     RAISE NOTICE '=====';
cursseurs$#     RAISE NOTICE '                RAPPORT DE PAIE PAR DEPARTEMENT                ';
cursseurs$#     RAISE NOTICE '=====';
cursseurs$#     FOR r_dept IN cur_dept LOOP
cursseurs$#         RAISE NOTICE '--- DEPARTEMENT % : % ---', r_dept.dept_id, UPPER(r_dept.nom_dept);
cursseurs$#         v_total_dept := 0; v_max_sal := 0; v_nb_emp := 0;
cursseurs$#         FOR r_emp IN cur_emp(r_dept.dept_id) LOOP
cursseurs$#             v_nb_emp := v_nb_emp + 1; v_nb_global := v_nb_global + 1;
cursseurs$#             v_total_dept := v_total_dept + r_emp.salaire + r_emp.prime;
cursseurs$#             v_total_global := v_total_global + r_emp.salaire + r_emp.prime;
cursseurs$#             IF r_emp.salaire > v_max_sal THEN v_max_sal := r_emp.salaire; END IF;
cursseurs$#             RAISE NOTICE '    * % | Salaire : % € | Prime : % € | Total : % €',
cursseurs$#                 RPAD(r_emp.nom, 10), r_emp.salaire, r_emp.prime, (r_emp.salaire + r_emp.prime);
cursseurs$#         END LOOP;
cursseurs$#         IF v_nb_emp > 0 THEN
cursseurs$#             RAISE NOTICE '    > Total paie département : % €', v_total_dept;
cursseurs$#             RAISE NOTICE '    > Salaire maximum          : % €', v_max_sal;
cursseurs$#         ELSE
cursseurs$#             RAISE NOTICE '    [INFO] Aucun employé affecté à ce département.';
cursseurs$#         END IF;
cursseurs$#     END LOOP;
cursseurs$#     RAISE NOTICE '=====';
cursseurs$#     RAISE NOTICE 'TOTAL GENERAL DE LA PAIE : % € (sur % employés)', v_total_global, v_nb_global;
cursseurs$#     RAISE NOTICE '=====';
cursseurs$# END; $$;

```

Résultat :

```

NOTICE: =====
NOTICE:                RAPPORT DE PAIE PAR DEPARTEMENT               
NOTICE: =====
NOTICE: --- DEPARTEMENT 10 : COMPTA ---
NOTICE:    * Martin   | Salaire : 3200.00 € | Prime : 500.00 € | Total : 3700.00 €
NOTICE:    * Petit    | Salaire : 2800.00 € | Prime : 300.00 € | Total : 3100.00 €
NOTICE:    * Durand   | Salaire : 2800.00 € | Prime : 140.00 € | Total : 2940.00 €
NOTICE:    * Leroy    | Salaire : 1900.00 € | Prime : 95.00 €  | Total : 1995.00 €
NOTICE:    -----
NOTICE:    > Total paie département : 11735.00 €
NOTICE:    > Salaire maximum          : 3200.00 €
NOTICE: --- DEPARTEMENT 20 : RH ---
NOTICE:    * Moreau   | Salaire : 2500.00 € | Prime : 200.00 € | Total : 2700.00 €
NOTICE:    * Simon    | Salaire : 2100.00 € | Prime : 0 €      | Total : 2100.00 €
NOTICE:    * Laurent  | Salaire : 1700.00 € | Prime : 100.00 € | Total : 1800.00 €
NOTICE:    -----
NOTICE:    > Total paie département : 6600.00 €
NOTICE:    > Salaire maximum          : 2500.00 €
NOTICE: --- DEPARTEMENT 30 : IT ---
NOTICE:    * Garcia   | Salaire : 4100.00 € | Prime : 900.00 € | Total : 5000.00 €
NOTICE:    * Roux     | Salaire : 3600.00 € | Prime : 0 €      | Total : 3600.00 €
NOTICE:    * Fournier | Salaire : 3000.00 € | Prime : 400.00 € | Total : 3400.00 €
NOTICE:    * Girard   | Salaire : 2200.00 € | Prime : 0 €      | Total : 2200.00 €
NOTICE:    * Bonnet   | Salaire : 1800.00 € | Prime : 0 €      | Total : 1800.00 €
NOTICE:    -----
NOTICE:    > Total paie département : 16000.00 €
NOTICE:    > Salaire maximum          : 4100.00 €
NOTICE: --- DEPARTEMENT 40 : ARCHIVES ---
NOTICE:    [INFO] Aucun employé affecté à ce département.
NOTICE: =====
NOTICE: TOTAL GENERAL DE LA PAIE : 34335.00 € (sur 12 employés)
NOTICE: =====
DO

```

Le rapport est généré de manière claire et structurée. La compta totalise 11 735.00 € avec un salaire maximum de 3 200.00 €, les RH totalisent 6 600.00 €, l'IT totalise 16 000.00 € avec un max de 4 100.00 €, et les Archives sont signalées comme vides. Le total général de la masse salariale de l'entreprise s'élève exactement à 34 335.00 € sur 12 employés.

Synthèse et questions de cours

À la fin du sujet de TD (page 9), trois questions de synthèse nous sont posées pour valider notre compréhension des curseurs :

1. Pourquoi et quand utiliser un curseur explicite plutôt qu'un simple SELECT INTO ?

On utilise un curseur explicite dès qu'une requête SQL est susceptible de renvoyer plusieurs lignes, ou 0 ligne, et que l'on veut traiter chaque enregistrement un par un. Le simple SELECT INTO est réservé aux requêtes qui renvoient obligatoirement une et une seule ligne : s'il renvoie 0 ligne il déclenche l'erreur NO_DATA_FOUND, et s'il en renvoie plusieurs il déclenche l'erreur TOO_MANY_ROWS. Le curseur explicite offre donc un contrôle total sur l'itération sans bloquer le programme.

2. Quelle a été l'erreur la plus surprenante rencontrée durant les tests ?

L'erreur la plus surprenante a été l'oubli du FETCH à l'intérieur de la boucle WHILE dans l'exercice 4 : le curseur restait bloqué indéfiniment sur la première ligne lue et provoquait une boucle infinie. De même, l'erreur CURSOR_ALREADY_OPEN montre bien que PostgreSQL refuse strictement d'ouvrir un curseur déjà actif sans qu'il ait été préalablement fermé.

3. Quel problème technique a été rencontré et comment a-t-il été résolu ?

Le principal problème technique a été la division par zéro lors du calcul de la moyenne des salaires sur le département 40 (Archives) dans l'exercice 5, car ce département ne contient aucun employé. Pour résoudre ce problème, nous avons ajouté une condition préalable vérifiant si le compteur d'employés est supérieur à 0 avant de faire la division. Si le compteur vaut 0, nous affichons un message informatif indiquant qu'aucun employé n'a été trouvé.

4) Conclusion

Ce TD 7 nous a permis de comprendre en profondeur l'utilité et le fonctionnement des curseurs en PL/pgSQL. Nous avons vu la différence entre les curseurs implicites et explicites, le suivi de leurs 4 attributs, les pièges à éviter et les bonnes pratiques avec la boucle FOR. Enfin, les curseurs paramétrés et l'option FOR UPDATE nous montrent toute la souplesse offerte pour manipuler les données ligne par ligne dans une base de données relationnelle.